

```

import re
import string
import joblib
import numpy as np
import pandas as pd

from scipy.sparse import hstack, csr_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import nltk
from nltk.corpus import stopwords
from nltk.sentiment import SentimentIntensityAnalyzer

nltk.download("stopwords")
nltk.download("vader_lexicon")

STOP_WORDS = set(stopwords.words("english"))
sia = SentimentIntensityAnalyzer()

SUSPICIOUS_TERMS = {
    "shocking", "breaking", "exclusive", "secret", "truth", "exposed",
    "banned", "must", "urgent", "alert", "scam", "fraud", "hoax",
    "miracle", "conspiracy", "propaganda", "fake", "viral"
}

COLUMN_NAMES = [
    "id", "label", "statement", "subjects", "speaker", "job_title",
    "state_info", "party_affiliation", "barely_true_counts",
    "false_counts", "half_true_counts", "mostly_true_counts",
    "pants_on_fire_counts", "context"
]

LABEL_MAP = {
    "pants-fire": 1,
    "false": 1,
    "barely-true": 1,
    "half-true": 0,
    "mostly-true": 0,
    "true": 0
}

def load_tsv_file(path):
    df = pd.read_csv(path, sep="\t", header=None, names=COLUMN_NAMES)
    df = df[["label", "statement"]].copy()
    df["label"] = df["label"].map(LABEL_MAP)
    df = df.dropna(subset=["label", "statement"]).reset_index(drop=True)
    return df

def load_data(train_path="train.tsv", valid_path="valid.tsv", test_path="test.tsv"):
    train_df = load_tsv_file(train_path)
    valid_df = load_tsv_file(valid_path)
    test_df = load_tsv_file(test_path)
    return train_df, valid_df, test_df

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+|https\S+", " ", text)
    text = re.sub(r"\d+", " ", text)
    text = text.translate(str.maketrans("", "", string.punctuation))
    text = re.sub(r"\s+", " ", text).strip()
    return text

def extract_manual_features(text):

```

```

raw = str(text)
cleaned = clean_text(raw)
tokens = cleaned.split()

word_count = len(tokens)
char_count = len(raw)
unique_ratio = len(set(tokens)) / (word_count + 1)
stopword_ratio = sum(1 for t in tokens if t in STOP_WORDS) / (word_count + 1)
uppercase_ratio = sum(1 for ch in raw if ch.isupper()) / (len(raw) + 1)

exclamation_count = raw.count("!")
question_count = raw.count("?")
digit_ratio = sum(1 for ch in raw if ch.isdigit()) / (len(raw) + 1)

suspicious_count = sum(1 for t in tokens if t in SUSPICIOUS_TERMS)
suspicious_ratio = suspicious_count / (word_count + 1)

sentiment = sia.polarity_scores(raw)
polarity_compound = sentiment["compound"]
polarity_pos = sentiment["pos"]
polarity_neg = sentiment["neg"]
polarity_neu = sentiment["neu"]

avg_word_len = np.mean([len(t) for t in tokens]) if tokens else 0

return [
    word_count,
    char_count,
    unique_ratio,
    stopwords_ratio,
    uppercase_ratio,
    exclamation_count,
    question_count,
    digit_ratio,
    suspicious_ratio,
    polarity_compound,
    polarity_pos,
    polarity_neg,
    polarity_neu,
    avg_word_len
]

def build_feature_matrix(df, vectorizer=None, fit=True):
    df = df.copy()
    df["clean_statement"] = df["statement"].apply(clean_text)
    manual_features = np.array(df["statement"].apply(extract_manual_features).tolist())

    if fit:
        vectorizer = TfidfVectorizer(
            max_features=5000,
            ngram_range=(1, 2),
            min_df=2,
            max_df=0.95
        )
        tfidf_matrix = vectorizer.fit_transform(df["clean_statement"])
    else:
        tfidf_matrix = vectorizer.transform(df["clean_statement"])

    manual_sparse = csr_matrix(manual_features)
    X = hstack([tfidf_matrix, manual_sparse])

    return X, vectorizer

def get_feature_names(vectorizer):
    manual_names = [
        "word_count", "char_count", "unique_ratio", "stopword_ratio",
        "uppercase_ratio", "exclamation_count", "question_count",
        "digit_ratio", "suspicious_ratio", "polarity_compound",

```

```

        "polarity_pos", "polarity_neg", "polarity_neu", "avg_word_len"
    ]
    tfidf_names = list(vectorizer.get_feature_names_out())
    return tfidf_names + manual_names

def train_model():
    train_df, valid_df, test_df = load_data()

    train_valid_df = pd.concat([train_df, valid_df], ignore_index=True)

    X_train, vectorizer = build_feature_matrix(train_valid_df, fit=True)
    y_train = train_valid_df["label"].values

    X_test, _ = build_feature_matrix(test_df, vectorizer=vectorizer, fit=False)
    y_test = test_df["label"].values

    model = RandomForestClassifier(
        n_estimators=300,
        max_depth=30,
        min_samples_split=5,
        min_samples_leaf=2,
        random_state=42,
        n_jobs=-1,
        class_weight="balanced"
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[: , 1]

    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("\nClassification Report:\n", classification_report(y_test, y_pred))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

    joblib.dump(model, "rf_liar_model.pkl")
    joblib.dump(vectorizer, "tfidf_liar_vectorizer.pkl")

    return model, vectorizer, test_df, y_test, y_pred, y_prob

def explain_prediction(text, model, vectorizer, top_k=10):
    temp_df = pd.DataFrame({"statement": [text]})
    X_new, _ = build_feature_matrix(temp_df, vectorizer=vectorizer, fit=False)

    pred = model.predict(X_new)[0]
    prob = model.predict_proba(X_new)[0]
    confidence = round(float(max(prob)) * 100, 2)

    feature_names = get_feature_names(vectorizer)
    importances = model.feature_importances_

    row = X_new.toarray()[0]
    active_idx = np.where(row > 0)[0]

    contributions = []
    for idx in active_idx:
        score = row[idx] * importances[idx]
        contributions.append((feature_names[idx], score))

    contributions = sorted(contributions, key=lambda x: x[1], reverse=True)[:top_k]
    suspicious_words = [name for name, score in contributions if name.isalpha()]

    label = "FAKE" if pred == 1 else "REAL"

    explanation = {
        "label": label,
        "confidence": confidence,
        "top_features": contributions,

```

```

        "highlight_words": suspicious_words
    }
    return explanation

def highlight_text(text, words):
    result = text
    for word in sorted(set(words), key=len, reverse=True):
        pattern = re.compile(rf"\b({re.escape(word)})\b", re.IGNORECASE)
        result = pattern.sub(r"[\1]", result)
    return result

if __name__ == "__main__":
    model, vectorizer, test_df, y_test, y_pred, y_prob = train_model()

    sample_text = "Breaking news! This shocking claim proves the government is hiding the truth fr
    explanation = explain_prediction(sample_text, model, vectorizer)

    print("\nPrediction:", explanation["label"])
    print("Confidence:", explanation["confidence"], "%")
    print("Top Features:", explanation["top_features"])
    print("Highlighted Text:")
    print(highlight_text(sample_text, explanation["highlight_words"]))

```

```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!
Accuracy: 0.5998421468034728

```

Classification Report:

	precision	recall	f1-score	support
0	0.66	0.61	0.63	714
1	0.54	0.59	0.56	553
accuracy			0.60	1267
macro avg	0.60	0.60	0.60	1267
weighted avg	0.61	0.60	0.60	1267

Confusion Matrix:

```

[[432 282]
 [225 328]]

```

Prediction: FAKE

Confidence: 55.29 %

Top Features: [('char_count', np.float64(1.0146584341455476)), ('word_count', np.float64(0.1295357

Highlighted Text:

Breaking news! This shocking claim proves [[the]] government [[is]] hiding [[the]] truth from peop

